

Carlos Coronel • Steven Morris

Database Systems

Design, Implementation, & Management



National Economics University Faculty of Mathematical Economics

Database System Design, Implementation And Management Course – Advanced Data Modeling

- ❑ Instructor: Duc Minh Vu (MFE)
- ❑ Email: minhvd [at] neu.edu.vn



Learning Objectives

- ❑ **5-1** Describe the main extended entity relationship (EER) model constructs and how they are represented in ERDs and EERDs
- ❑ **5-2** Use entity clusters to represent multiple entities and relationships in an entity relationship diagram (ERD)
- ❑ **5-3** Describe the characteristics of good primary keys and how to select them
- ❑ **5-4** Apply flexible solutions for special data-modeling cases



5-1 The Extended Entity Relationship Model

Figure 5.1 Nulls Created by Unique Attributes

Database name: Ch05_AirCo

EMP_NUM	EMP_LNAME	EMP_FNAME	EMP_INITIAL	EMP_LICENSE	EMP_RATINGS	EMP_MED_TYPE	EMP_HIRE_DATE
100	Kolmycz	Xavier	T				15-Mar-92
101	Lewis	Marcos		ATP	SEL/MEL/Instr/CFII	1	25-Apr-93
102	Vandam	Jean					20-Dec-97
103	Jones	Victoria	R				28-Aug-07
104	Lange	Edith		ATP	SEL/MEL/Instr	1	20-Oct-01
105	Williams	Gabriel	U	COM	SEL/MEL/Instr/CFI	2	08-Nov-01
106	Duzak	Mario		COM	SEL/MEL/Instr	2	05-Jan-08
107	Diante	Venite	L				02-Jul-01
108	Wiesenbach	Joni					18-Nov-99
109	Travis	Brett	T	COM	SEL/MEL/SES/Instr/CFII	1	14-Apr-05
110	Genkazi	Stan					01-Dec-07

An EMPLOYEE table with many empty and null cells because of many types of employees (pilot, mechanic, account, etc.)



5-1 The Extended Entity Relationship Model

□ extended entity relationship model (EERM)

- Enhanced entity relationship model; the result of adding more semantic constructs, such as entity supertypes, entity subtypes, and entity clustering, to the original entity relationship (ER) model.

□ EER diagram (EERD)

- The entity relationship diagram resulting from the application of extended entity relationship concepts that provide additional semantic content in the ER model.



5-1a Entity Supertypes and Subtypes

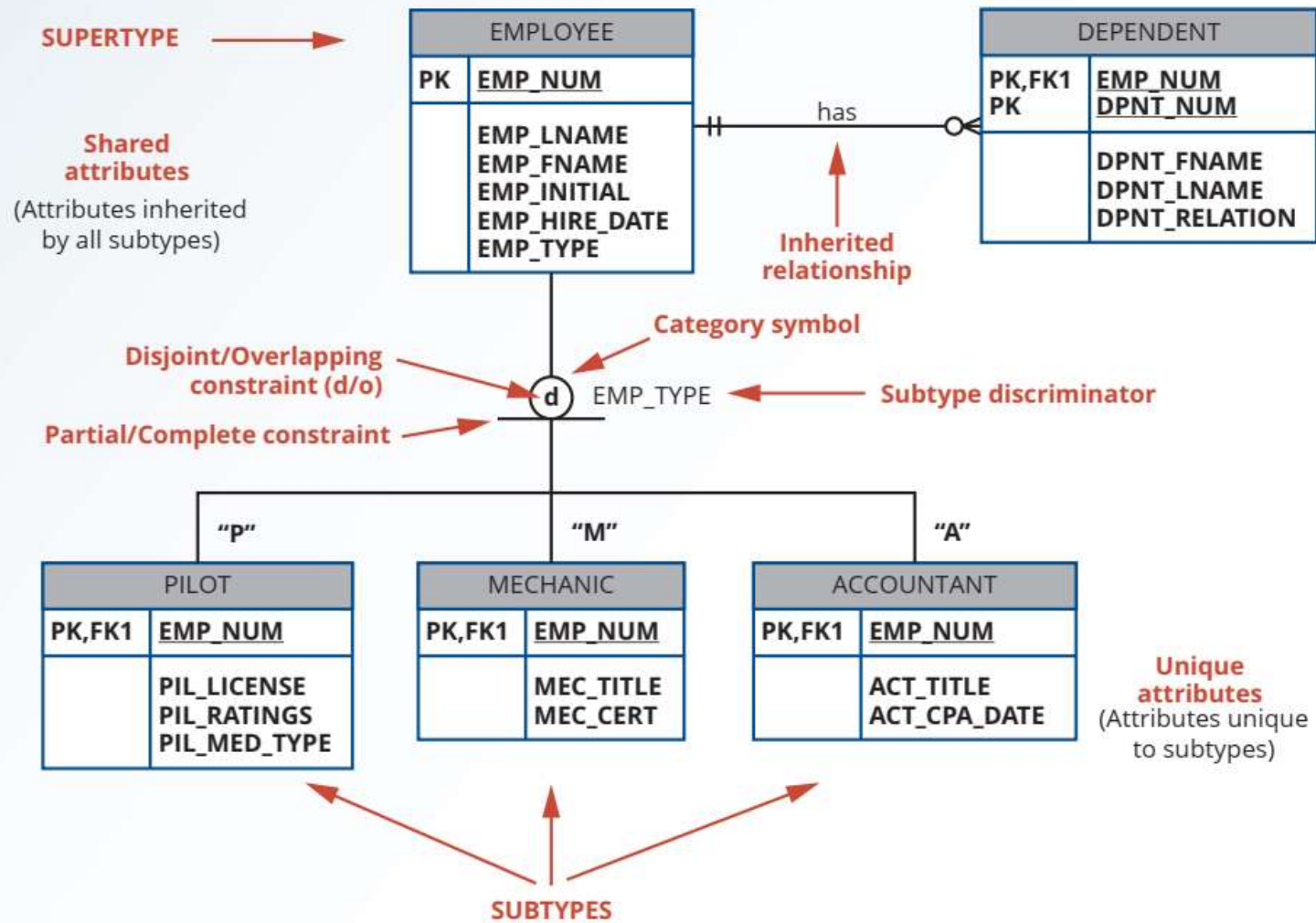
- ❑ Similar to base class and derived class in OOP.
- ❑ **entity supertype**: generic entity type that contains the common characteristics of entity subtypes.
 - EMPLOYEE
- ❑ **entity subtype**: the subtypes contain the unique characteristics of each entity.
 - PILOT, MECHANIC, ACCOUNTANT
- ❑ Two criteria help the designer determine when to use subtypes and supertypes:
 - There must be different, identifiable kinds or types of the entity in the user's environment.
 - The different kinds or types of instances should each have one or more attributes that are unique to that kind or type of instance.



5-1b Specialization Hierarchy

- ❑ Entity supertypes and subtypes are organized in a **specialization hierarchy**, which depicts the arrangement of higher-level entity supertypes (parent entities) and lower-level entity subtypes (child entities).
 - The specialization hierarchy formed by an EMPLOYEE supertype and three entity subtypes—PILOT, MECHANIC, and ACCOUNTANT.
- ❑ “Is-a” relation: a pilot *is an* employee, a mechanic *is an* employee.
- ❑ A specialization hierarchy provides the means to:
 - Support attribute inheritance.
 - Define a special supertype attribute known as the subtype discriminator.
 - Define disjoint or overlapping constraints and complete or partial constraints.

Figure 5.2 A Specialization Hierarchy





5-1c Inheritance

- ❑ The property of **inheritance** enables an entity subtype to inherit the attributes and relationships of the supertype.
- ❑ *Entity subtypes inherit their primary key attribute from their supertype*
- ❑ Entity subtypes inherit all relationships in which the supertype entity participates

Figure 5.3 The Employee-Pilot Supertype-Subtype Relationship

Table name: EMPLOYEE

EMP_NUM	EMP_LNAME	EMP_FNAME	EMP_INITIAL	EMP_HIRE_DATE	EMP_TYPE
100	Kolmycz	Xavier	T	15-Mar-92	
101	Lewis	Marcos		25-Apr-93	P
102	Vandam	Jean		20-Dec-97	A
103	Jones	Victoria	R	28-Aug-07	
104	Lange	Edith		20-Oct-01	P
105	Williams	Gabriel	U	08-Nov-01	P
106	Duzak	Mario		05-Jan-08	P
107	Diante	Venite	L	02-Jul-01	M
108	Wiesenbach	Joni		18-Nov-99	M
109	Travis	Brett	T	14-Apr-05	P
110	Genkazi	Stan		01-Dec-07	A

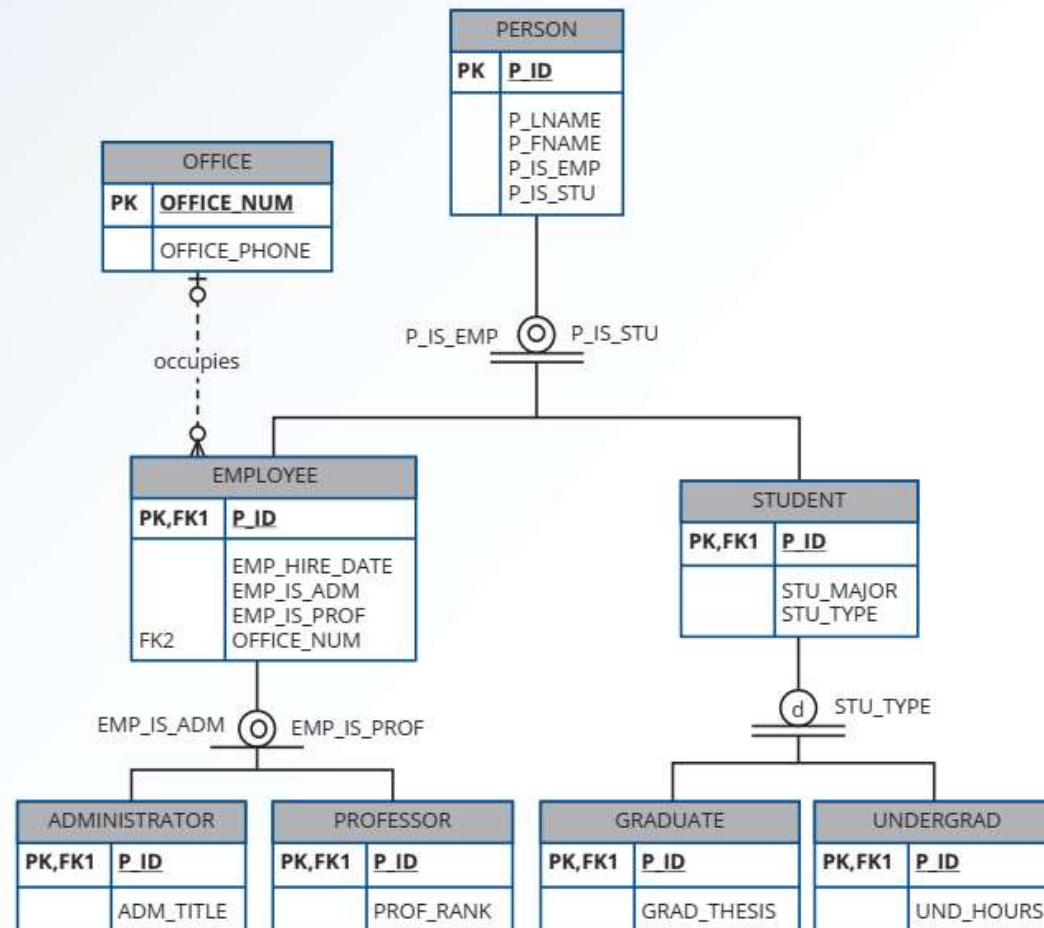
Database name: Ch05_AirCo

Table name: PILOT

EMP_NUM	PIL_LICENSE	PIL_RATINGS	PIL_MED_TYPE
101	ATP	SEL/MEL/Instr/CFII	1
104	ATP	SEL/MEL/Instr	1
105	COM	SEL/MEL/Instr/CFI	2
106	COM	SEL/MEL/Instr	2
109	COM	SEL/MEL/SES/Instr/CFII	1

At the implementation level, the supertype and its subtype(s) depicted in the specialization hierarchy maintain a 1:1 relationship. For example, the specialization hierarchy lets you replace the undesirable EMPLOYEE table structure in Figure 5.1 with two tables—one representing the supertype EMPLOYEE and the other representing the subtype PILOT.

Figure 5.4 Specialization Hierarchy with Overlapping Subtypes





5-1d Subtype Discriminator

- A **subtype discriminator** is the attribute in the supertype entity that determines to which subtype the supertype occurrence is related
- The subtype discriminator is the employee type (EMP_TYPE)
 - the supertype is related to a PILOT subtype if the EMP_TYPE has a value of “P.” If the EMP_TYPE value is “M,” the supertype is related to a MECHANIC subtype. If the EMP_TYPE value is “A,” the supertype is related to the ACCOUNTANT subtype.



5-1e Disjoint and Overlapping Constraints

- ❑ An entity supertype can have disjoint or overlapping entity subtypes
 - an employee can be a pilot, a mechanic, or an accountant
- ❑ **Disjoint subtypes**, or **nonoverlapping subtypes**, are subtypes that contain a *unique* subset of the supertype entity set - each entity instance of the supertype can appear in only one of the subtypes
 - Employee cannot be pilot and accountant at the same time
- ❑ **Overlapping subtypes** are subtypes that contain nonunique subsets of the supertype entity set; that is, each entity instance of the supertype may appear in more than one subtype



disjoint subtypes

In a specialization hierarchy, these are unique and nonoverlapping subtype entity sets.

nonoverlapping subtypes

See *disjoint subtypes*.

overlapping subtypes

In a specialization hierarchy, a condition in which each entity instance (row) of the supertype can appear in more than one subtype.

Figure 5.5 Disjoint and Overlapping Subtypes

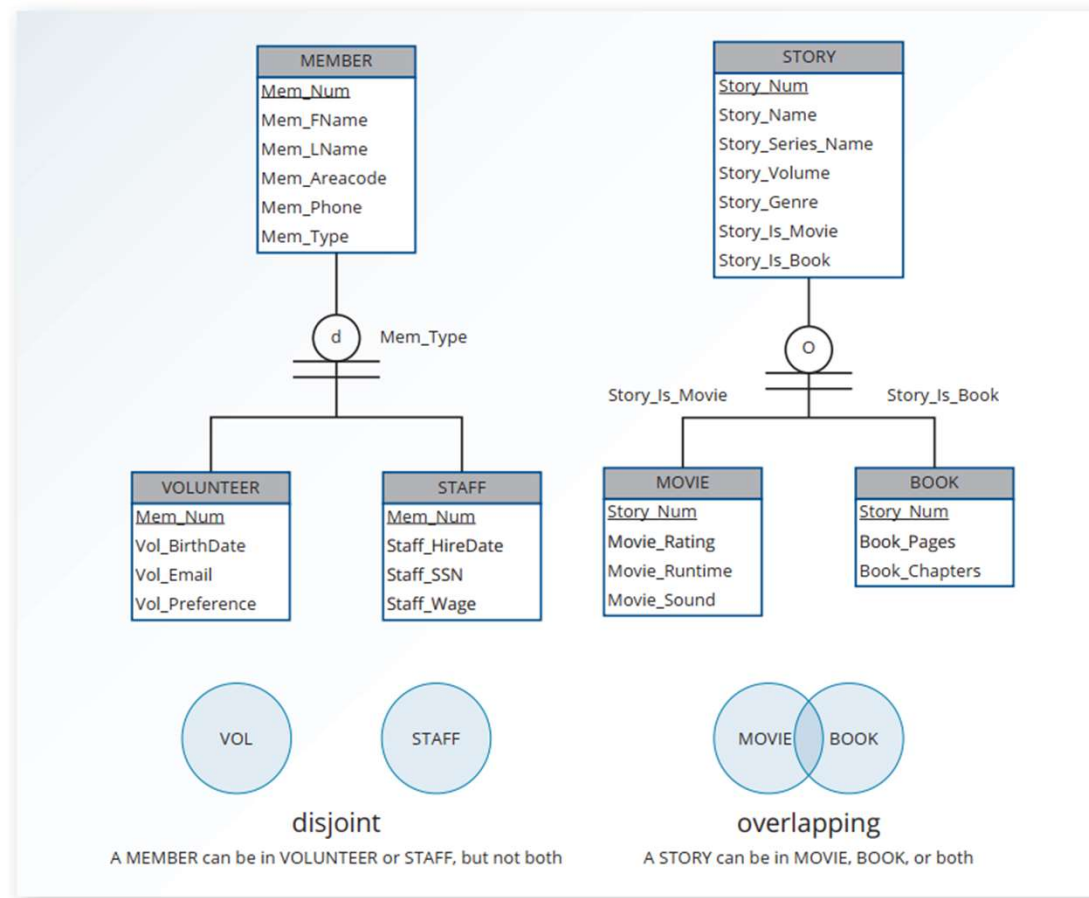




Table 5.1 Discriminator Attributes with Overlapping Subtypes

Discriminator Attributes		Comment
Professor	Administrator	
Y	N	The Employee is a member of the Professor subtype.
N	Y	The Employee is a member of the Administrator subtype.
Y	Y	The Employee is both a Professor and an Administrator.
N	N	The Employee is not required to be a member of any subtype. Only possible if the hierarchy exhibits the partial completeness constraint (see next section).



5-1f Completeness Constraint

- ❑ The **completeness constraint** specifies whether each entity supertype occurrence must also be a member of at least one subtype.
- ❑ **Partial completeness** means that not every supertype occurrence is a member of a subtype; some supertype occurrences may not be members of any subtype.
- ❑ **Total completeness** means that every supertype occurrence must be a member of at least one subtype

completeness constraint

A constraint that specifies whether each entity supertype occurrence must also be a member of at least one subtype. The completeness constraint can be partial or total.

partial completeness

In a generalization or specialization hierarchy, a condition in which some supertype occurrences might not be members of any subtype.

total completeness

In a generalization or specialization hierarchy, a condition in which every supertype occurrence must be a member of at least one subtype.



Table 5.2 Specialization Hierarchy Constraint Scenarios

Type	Disjoint Constraint	Overlapping Constraint
Partial 	Supertype has optional subtypes. Subtype discriminator can be null. Subtype sets are unique.	Supertype has optional subtypes. Subtype discriminators can be null. Subtype sets are not unique.
Total 	Every supertype occurrence is a member of only one subtype. Subtype discriminator cannot be null. Subtype sets are unique.	Every supertype occurrence is a member of at least one subtype. Subtype discriminators cannot be null. Subtype sets are not unique.



5-1g Specialization and Generalization

- ❑ Various approaches to develop entity supertypes and subtypes.
 - Identify a regular entity and then identify all entity subtypes based on their distinguishing characteristics.
 - Identify multiple entity types and then later extract the common characteristics of those entities to create a higher-level supertype entity.
- ❑ **Specialization** is the top-down process of identifying lower-level, more specific entity subtypes from a higher-level entity supertype
- ❑ **Generalization** is the bottom-up process of identifying a higher-level, more generic entity supertype from lower-level entity subtypes

specialization

In a specialization hierarchy, the grouping of unique attributes into a subtype entity.

generalization

In a specialization hierarchy, the grouping of common attributes into a supertype entity.



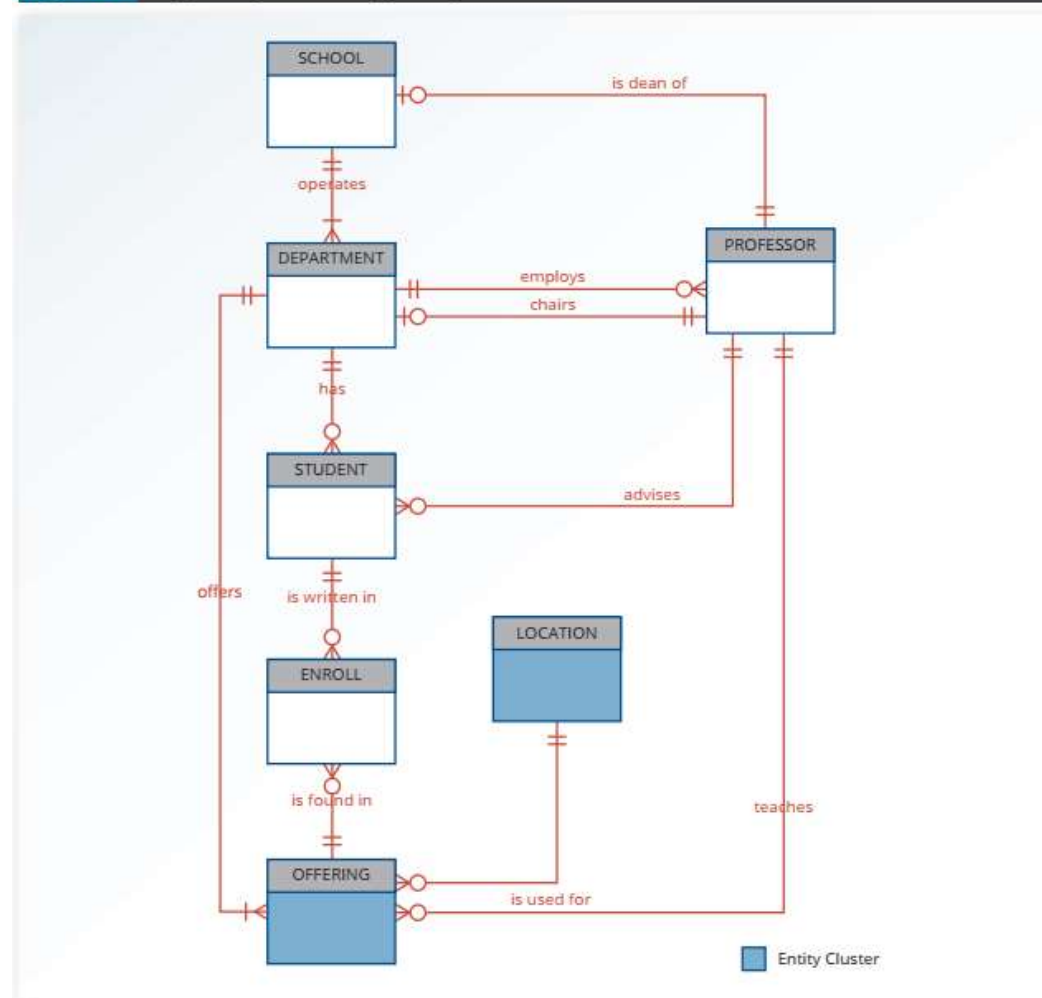
5-2 Entity Clustering



- ❑ An **entity cluster** is a “virtual” entity type used to represent multiple entities and relationships in the ERD.
- ❑ An entity cluster is formed by combining multiple interrelated entities into a single, abstract entity object.
- ❑ it is a “conceptual” entity used to represent multiple entities and relationships, with the purpose of simplifying the ERD and thus enhancing its readability.
- ❑ Tiny College Example:
 - OFFERING, which groups the SEMESTER, COURSE, and CLASS entities and relationships
 - LOCATION, which groups the ROOM and BUILDING entities and relationships



Figure 5.6 Tiny College ERD Using Entity Clusters





5-3 Entity Integrity: Selecting Primary Keys



5-3a Natural Keys and Primary Keys

- ❑ A **natural key** or **natural identifier** is a real-world, generally accepted identifier used to distinguish—that is, uniquely identify—real-world objects.
- ❑ As its name implies, a natural key is familiar to end users and forms part of their day-to-day business vocabulary.



5-3b Primary Key Guidelines

- ❑ A primary key is the attribute or combination of attributes that uniquely identifies entity instances in an entity set.
- ❑ Its main function is to uniquely identify an entity instance or row within a table
- ❑ Second, primary keys and foreign keys are used to implement relationships among entities



5-3c When to Use Composite Primary Keys

- ❑ As identifiers of composite entities, in which each primary key combination is allowed only once in the M:N relationship
- ❑ As identifiers of weak entities, in which the weak entity has a strong identifying relationship with the parent entity

Table 5.3 Desirable Primary Key Characteristics

PK Characteristic	Rationale
Unique values	The PK must uniquely identify each entity instance. A primary key must be able to guarantee unique values. It cannot contain nulls.
Nonintelligent	The PK should not have embedded semantic meaning other than to uniquely identify each entity instance. An attribute with embedded semantic meaning is probably better used as a descriptive characteristic of the entity than as an identifier. For example, a student ID of 650973 would be preferred over Smith, Martha L. as a primary key identifier.
No change over time	If an attribute has semantic meaning, it might be subject to updates, which is why names do not make good primary keys. If Vickie Smith is the primary key, what happens if she changes her name when she gets married? If a primary key is subject to change, the foreign key values must be updated, thus adding to the database workload. Furthermore, changing a primary key value means that you are basically changing the identity of an entity. In short, the PK should be permanent and unchangeable.
Preferably single-attribute	A primary key should have the minimum number of attributes possible (irreducible). Single-attribute primary keys are desirable but not required. Single-attribute primary keys simplify the implementation of foreign keys. Having multiple-attribute primary keys can cause primary keys of related entities to grow through the possible addition of many attributes, thus adding to the database workload and making (application) coding more cumbersome.
Preferably numeric	Unique values can be better managed when they are numeric because the database can use internal routines to implement a counter-style attribute that automatically increments values with the addition of each new row. In fact, most database systems include the ability to use special constructs, such as AutoNumber in Microsoft Access, sequence in Oracle, the autoincrement constraint in MySQL, or the identity property in MS SQL Server, to support self-incrementing primary key attributes.
Security-compliant	The selected primary key must not be composed of any attribute(s) that might be considered a security risk or violation. For example, using a Social Security number as a PK in an EMPLOYEE table is not a good idea.



5-3d When to Use Surrogate Primary Keys

- ❑ A **surrogate key** is a primary key created by the database designer to simplify the identification of entity instances.
- ❑ A system-assigned primary key, generally numeric and auto-incremented



5-4 Design Cases: Learning Flexible Database Design



5-4a Design Case 1: Implementing 1:1 Relationships

- 1:1 relationship between EMPLOYEE and DEPARTMENT based on the business rule “one EMPLOYEE is the manager of one DEPARTMENT, and one DEPARTMENT is managed by one EMPLOYEE.”
 - *Place a foreign key in both entities*
 - *Place a foreign key in one of the entities*

Figure 5.8 The 1:1 Relationship Between DEPARTMENT and EMPLOYEE

A One-to-One (1:1) Relationship:

An EMPLOYEE manages zero or one DEPARTMENT;
each DEPARTMENT is managed by one EMPLOYEE.

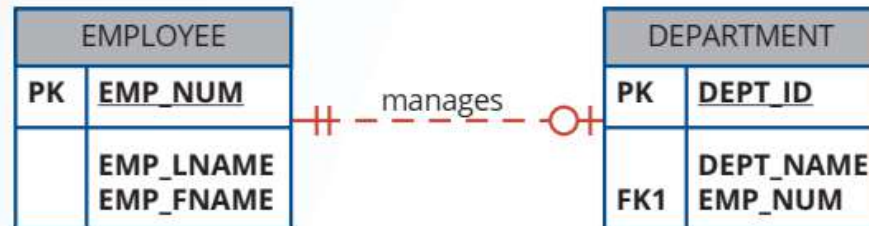




Table 5.5 Selection of Foreign Key in a 1:1 Relationship

Case	ER Relationship Constraints	Action
I	One side is mandatory and the other side is optional.	Place the PK of the entity on the mandatory side in the entity on the optional side as an FK, and make the FK mandatory.
II	Both sides are optional.	Select the FK that causes the fewest nulls, or place the FK in the entity in which the (relationship) role is played.
III	Both sides are mandatory.	See Case II, or consider revising your model to ensure that the two entities do not belong together in a single entity.



5-4b Design Case 2: Maintaining History of Time-Variant Data

- ❑ **time-variant data** refers to data whose values change over time and for which you *must* keep a history of the data changes.
- ❑ Some time-variant data is equivalent to having a multivalued attribute in your entity. To model this type of time-variant data, create a new entity in a 1:M relationship with the original entity.
 - This new entity will contain the new value, the date of the change, and any other attribute that is pertinent to the event being modeled
- ❑ Other time-variant data can turn a 1:M relationship into an M:N relationship.
 - Your data model includes data about the different departments in the organization and which employee manages each department. Each department is managed by only one employee and each employee can manage one department at most, then a 1:1 relationship would exist between EMPLOYEE and DEPARTMENT.

Figure 5.9 Maintaining Salary History

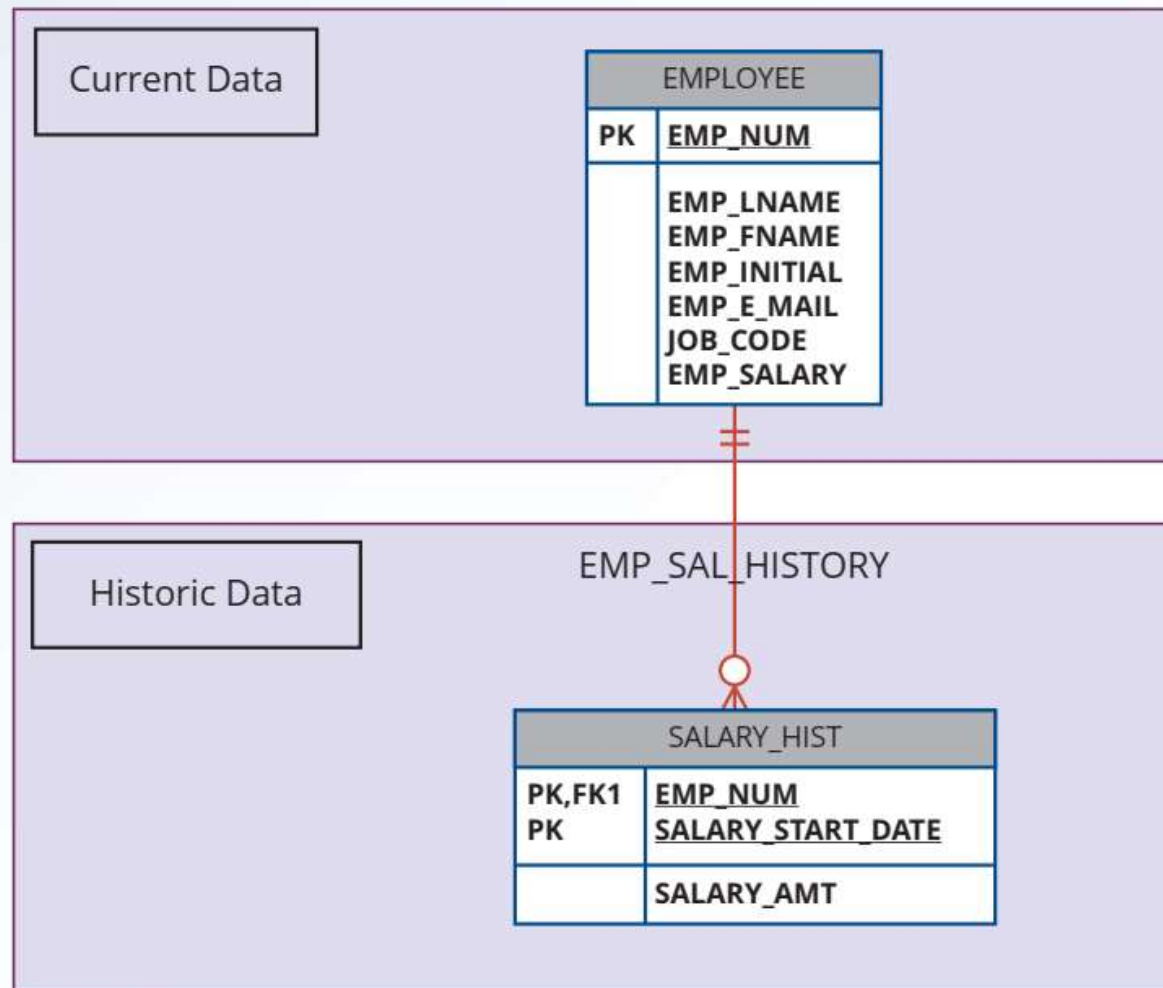


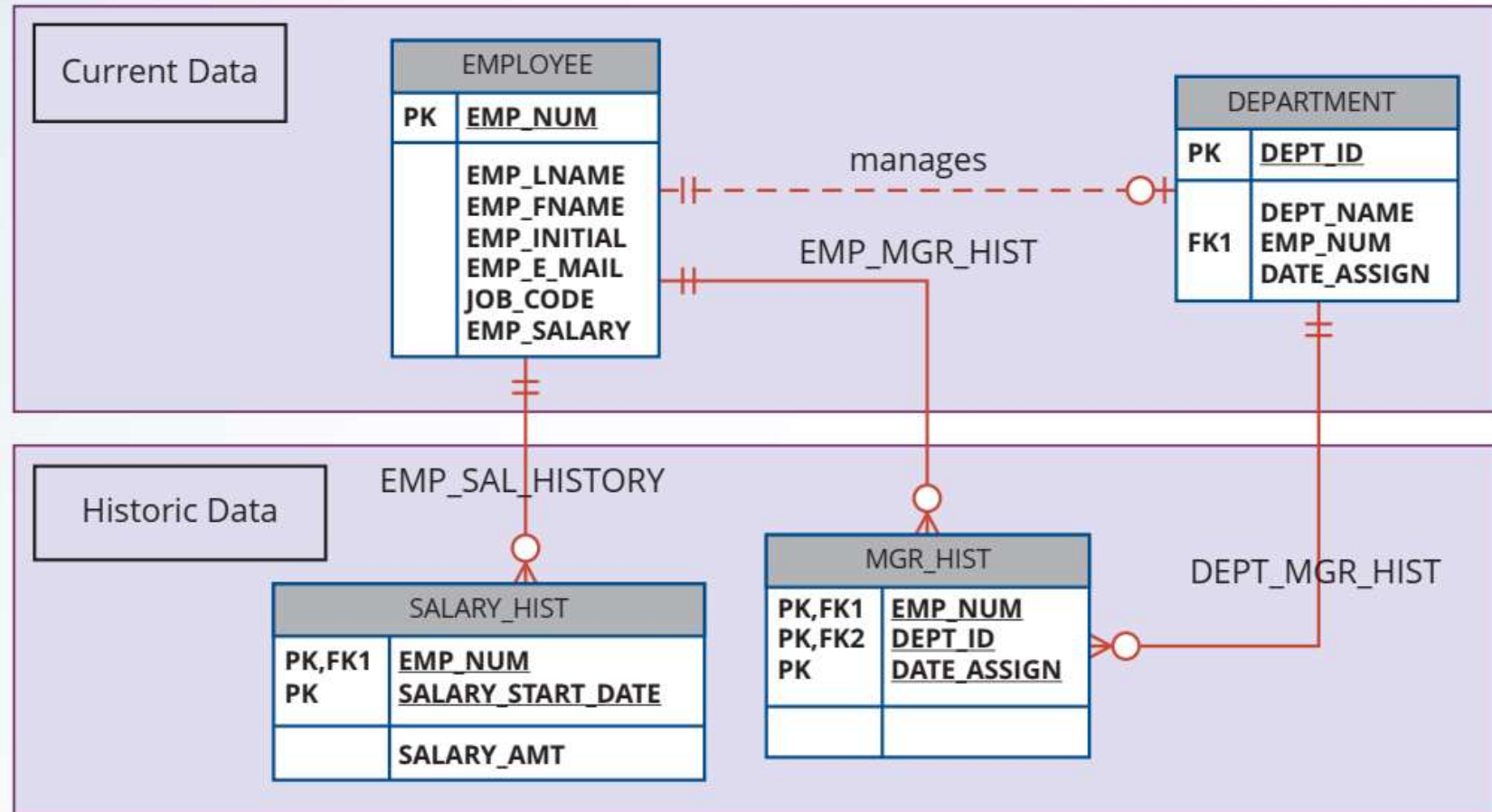
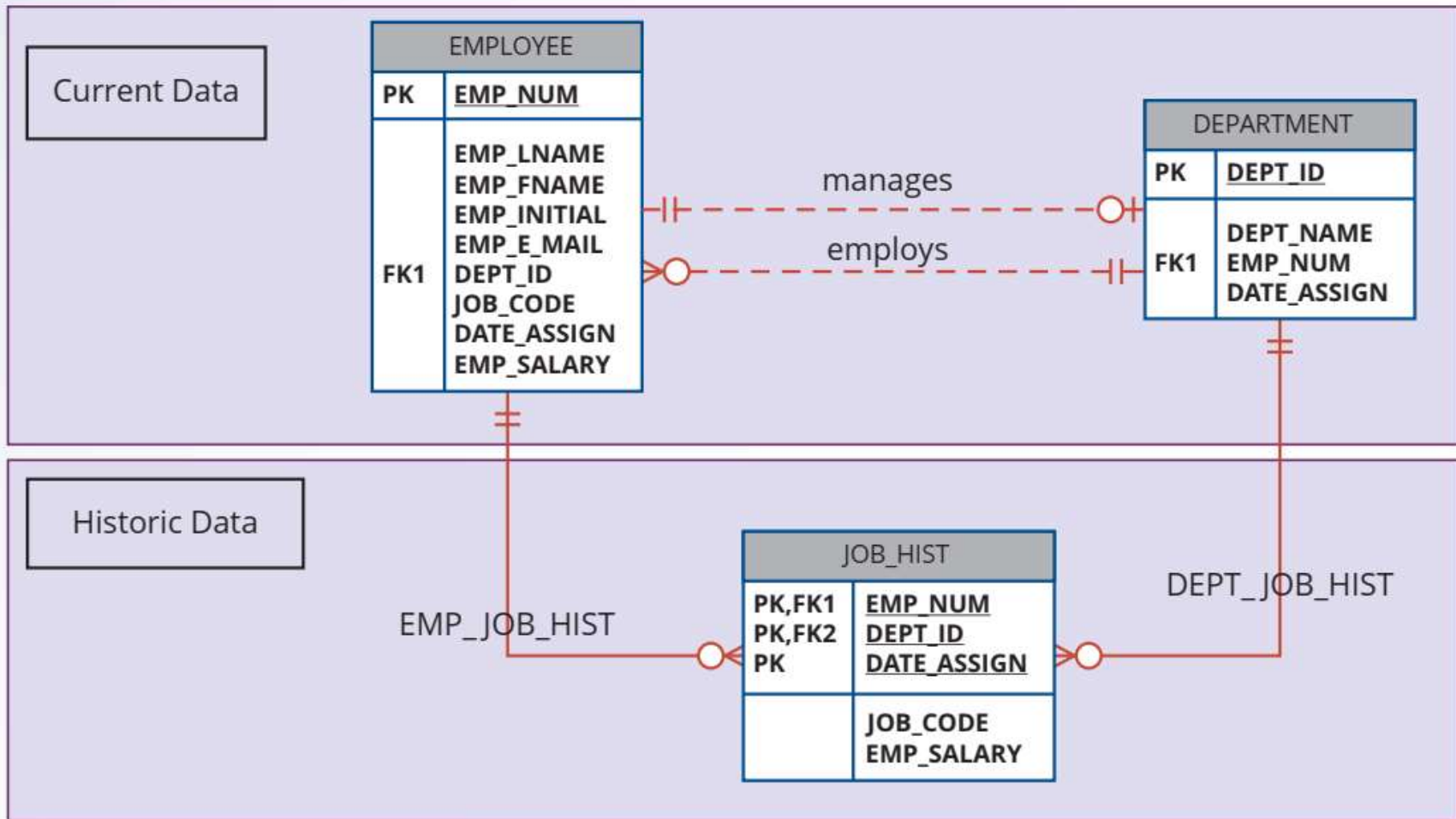
Figure 5.10 Maintaining Manager History


Figure 5.11 Maintaining Job History



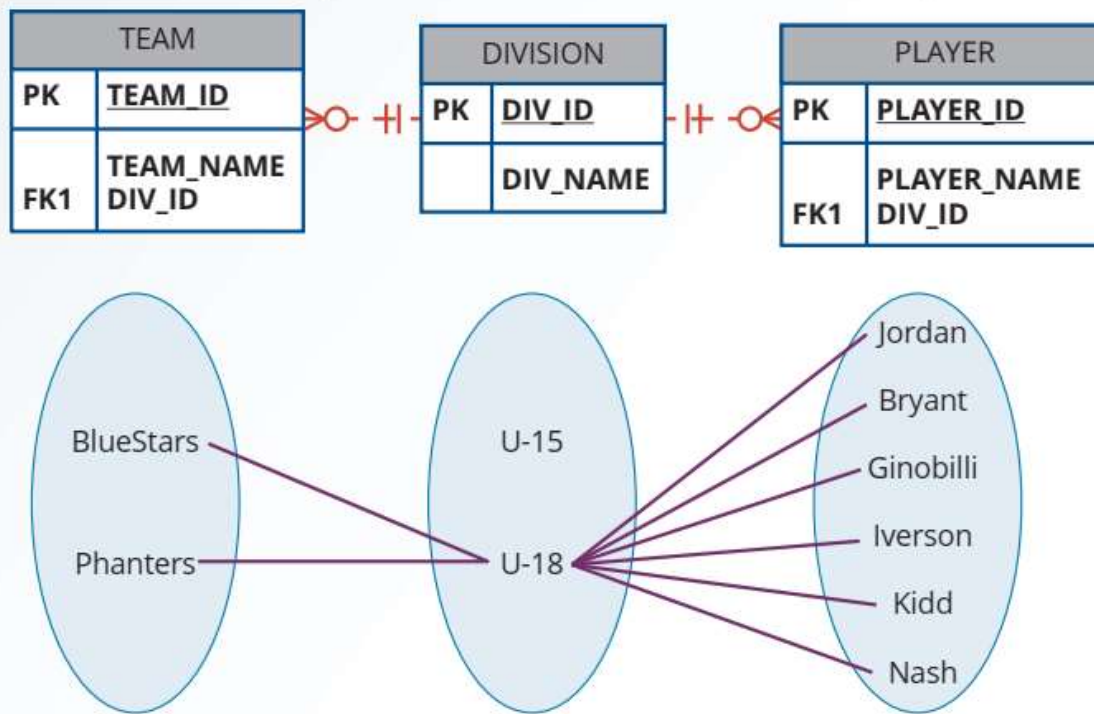


5-4c Design Case 3: Fan Traps

- ❑ A **design trap** occurs when a relationship is improperly or incompletely identified and is, therefore, represented in a way that is not consistent with the real world.
- ❑ A **fan trap** occurs when you have one entity in two 1:M relationships to other entities, thus producing an association among the other entities that is not expressed in the model.

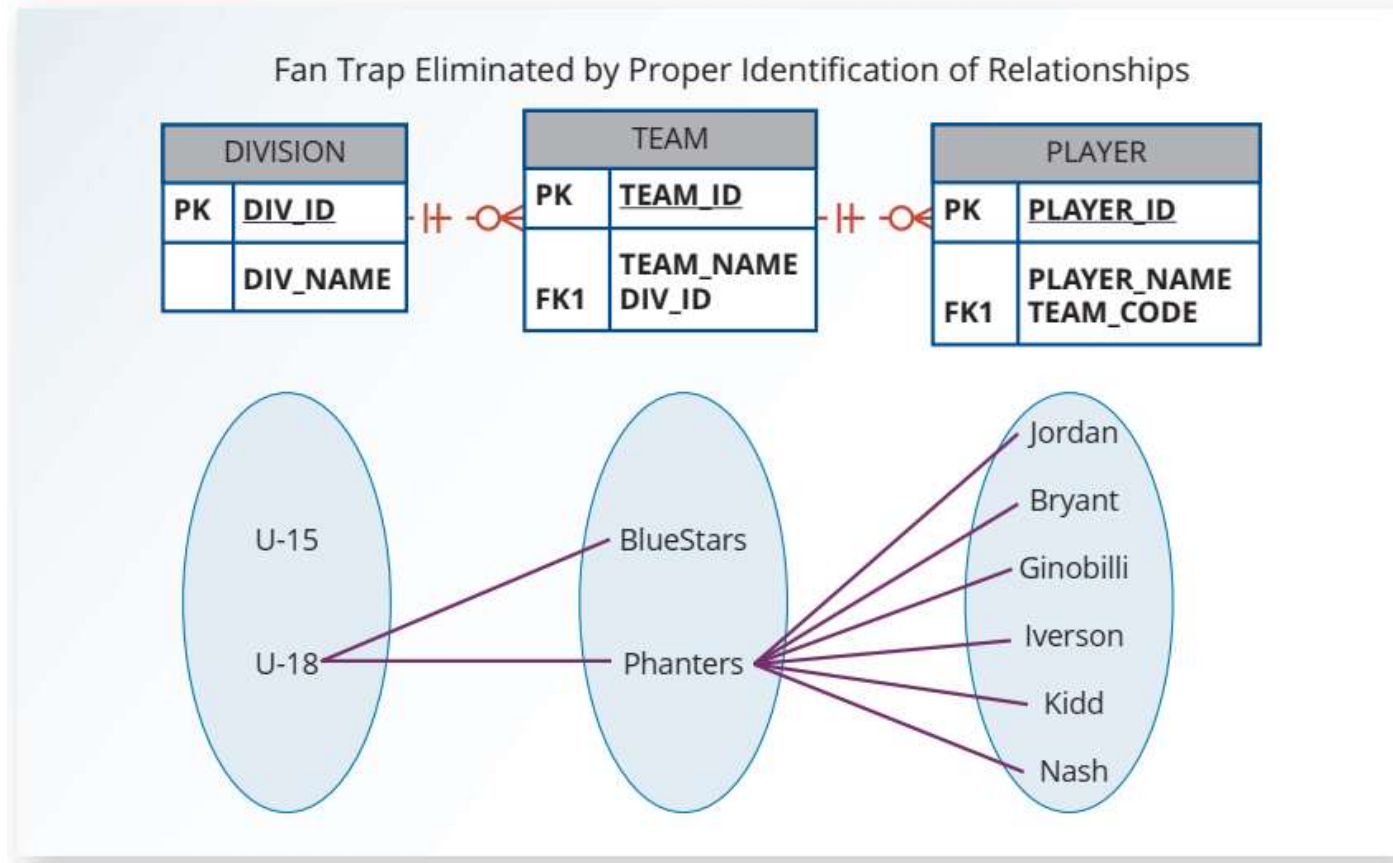
Figure 5.12 Incorrect ERD with Fan Trap Problem

Fan Trap Due to Misidentification of Relationships



there is no way to identify which players belong to which team.

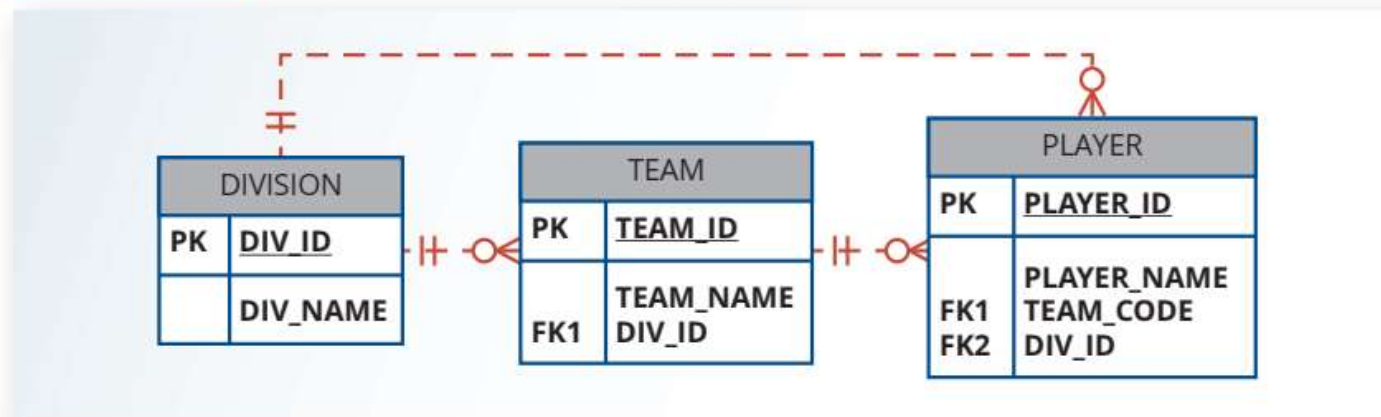
Figure 5.13 Corrected ERD after Removal of the Fan Trap



5-4d Design Case 4: Redundant Relationships

- ❑ Redundant relationships occur when there are multiple relationship paths between related entities.
- Transitive 1:M relationship between DIVISION and PLAYER through the TEAM entity set. Therefore, the relationship that connects DIVISION and PLAYER is redundant

Figure 5.14 A Redundant Relationship



Summary

- The extended entity relationship (EER) model adds semantics to the ER model via entity supertypes, subtypes, and clusters. An entity supertype is a generic entity type that is related to one or more entity subtypes.
- A specialization hierarchy depicts the arrangement and relationships between entity supertypes and entity subtypes. Inheritance means that an entity subtype inherits the attributes and relationships of the supertype. Subtypes can be disjoint or overlapping. A subtype discriminator is used to determine to which entity subtype the supertype occurrence is related. The subtypes can exhibit partial or total completeness. There are basically two approaches to developing a specialization hierarchy of entity supertypes and subtypes: specialization and generalization.
- An entity cluster is a “virtual” entity type used to represent multiple entities and relationships in the ERD. An entity cluster is formed by combining multiple inter-related entities and relationships into a single, abstract entity object.
- Natural keys are identifiers that exist in the real world. Natural keys sometimes make good primary keys, but not always. Primary keys must have unique values, they should be nonintelligent, they must not change over time, and they are preferably numeric and composed of a single attribute.
- Composite keys are useful to represent M:N relationships and weak (strong identifying) entities.
- Surrogate primary keys are useful when there is no natural key that makes a suitable primary key, when the primary key is a composite primary key with multiple data types, or when the primary key is too long to be usable.
- In a 1:1 relationship, place the PK of the mandatory entity as a foreign key in the optional entity, as an FK in the entity that causes the fewest nulls, or as an FK where the role is played.
- Time-variant data refers to data whose values change over time and require that you keep a history of data changes. To maintain the history of time-variant data, you must create an entity that contains the new value, the date of change, and any other time-relevant data. This entity maintains a 1:M relationship with the entity for which the history is to be maintained.
- A fan trap occurs when you have one entity in two 1:M relationships to other entities, and there is an association among the other entities that is not expressed in the model. Redundant relationships occur when there are multiple relationship paths between related entities. The main concern with redundant relationships is that they remain consistent across the model.



Key Terms

completeness constraint
design trap
disjoint subtype
EER diagram (EERD)
entity cluster
entity subtype
entity supertype

extended entity relationship model
(EERM)
fan trap
generalization
inheritance
natural key (natural identifier)
nonoverlapping subtype
overlapping subtype

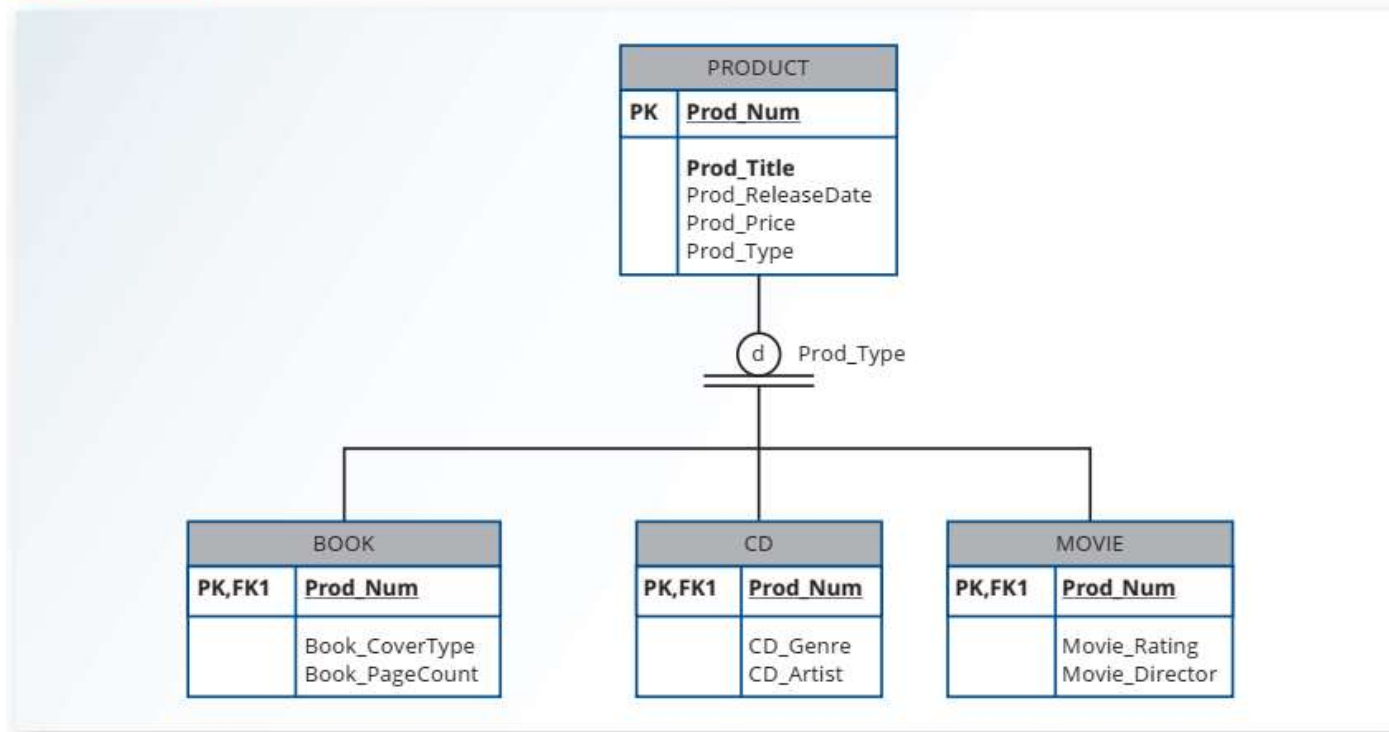
partial completeness
specialization
specialization hierarchy
subtype discriminator
surrogate key
time-variant data
total completeness



Review Questions

1. What is an entity supertype, and why is it used?
2. What kinds of data would you store in an entity subtype?
3. What is a specialization hierarchy?
4. What is a subtype discriminator? Give an example of its use.
5. What is an overlapping subtype? Give an example.
6. What is a disjoint subtype? Give an example.
7. What is the difference between partial completeness and total completeness?

Figure Q5.8 The PRODUCT Data Model





8. List all of the attributes of a movie.
9. According to the data model, is it required that every entity instance in the PRODUCT table be associated with an entity instance in the CD table? Why, or why not?
10. Is it possible for a book to appear in the BOOK table without appearing in the PRODUCT table? Why, or why not?
11. What is an entity cluster, and what advantages are derived from its use?
12. What primary key characteristics are considered desirable? Explain *why* each characteristic is considered desirable.
13. Under what circumstances are composite primary keys appropriate?
14. What is a surrogate primary key, and when would you use one?
15. When implementing a 1:1 relationship, where should you place the foreign key if one side is mandatory and one side is optional? Should the foreign key be mandatory or optional?
16. What is time-variant data, and how would you deal with such data from a database design point of view?
17. What is the most common design trap, and how does it occur?